Numerical Modelling of Orbital Motion and Gravitational Perturbations

Conor Lewis
23904082

March 2026

Abstract

Newtonian orbital motion provides a benchmark for numerical integration of nonlinear dynamical systems. This work evaluates the Euler–Cromer and second-order Runge–Kutta (RK2) schemes in reproducing Keplerian motion and examines how timestep size and additional gravitational perturbations affect the simulation. Energy and angular momentum conservation are used as diagnostics. Timestep refinement confirms numerical convergence, while timestep enlargement produces distortion consistent with truncation-error theory [2]. Extensions to a double-star field and a simplified Sun–Jupiter model illustrate the transition from integrable two-body motion to perturbed, non-periodic dynamics [3]. The results confirm theoretical expectations for numerical order, stability behaviour, and the dynamical consequences of broken rotational symmetry.

1 Introduction

The Newtonian two-body problem admits exact conic-section solutions governed by Kepler's laws [1]. When additional gravitating bodies are introduced, integrability is lost and numerical integration becomes essential. The reliability of such simulations depends on timestep selection, scheme order, and long-term stability properties.

This investigation addresses the question:

> How accurately can low-order numerical integrators reproduce Keplerian motion, and how do timestep size and gravitational perturbations influence a low order simulation?

Circular and elliptical two-body orbits are simulated using Euler–Cromer and RK2 methods. Accuracy is quantified via conservation diagnostics, convergence is tested through timestep refinement, and the force model is extended to multi-body configurations to probe the transition from integrable to perturbed dynamics.

2 Theory

The planar two-body equation of motion is

$$\ddot{\mathbf{r}} = -\frac{GM}{r^3}\mathbf{r}, \tag{1}$$

with $r = \sqrt{x^2 + y^2}$. In astronomical units and years, $GM = 4\pi^2$ so that Kepler's third law reduces to $T = a^{3/2}$ [1].

The specific energy and angular momentum are

$$E = \frac{1}{2}v^2 - \frac{GM}{r}, \qquad h = xv_y - yv_x. \tag{2}$$

For a central force, both are conserved. Deviations therefore provide quantitative measures of discretisation error. For non-symplectic integrators, bounded oscillatory energy error is expected, while long-term drift may appear for insufficient timestep resolution [2].

2.1 Three-Body Perturbation Framework

Introducing Jupiter produces a restricted three-body system. The potential becomes

$$V(\mathbf{r}, t) = -\frac{GM_\odot}{r} - \frac{GM_J}{|\mathbf{r} - \mathbf{r}_J(t)|}.$$

The force is no longer central about the origin, so angular momentum about the Sun is not conserved. The system is therefore non-integrable and admits secular evolution of orbital elements [3]. For weak perturbation ($GM_J \ll GM_\odot$), motion remains approximately Keplerian over short timescales but deviates progressively over many periods.

$$V(\mathbf{r}, t) = -\frac{GM_\odot}{r} - \frac{GM_J}{|\mathbf{r} - \mathbf{r}_J(t)|}.$$

Because the perturbing mass moves with time, the force field is no longer central with respect to the origin, and the angular momentum of the test particle about the Sun is no longer conserved For sufficiently weak perturbation ($GM_J \ll GM_\odot$), the motion remains approximately Keplerian over short timescales but deviates progressively over many orbital periods.

3 Computational Methods

3.1 Time Discretisation

A snapshot-based timestep was defined as

$$\Delta t = \frac{T}{N_{\text{snap}} N_{\text{between}}}, \tag{3}$$

ensuring that temporal resolution scales with the orbital period. For $N_{\text{snap}} = 60$ and $N_{\text{between}} = 20$, a one-year orbit corresponds to $\Delta t = 8.33 \times 10^{-4}$ yr, or 1200 integration steps per orbit.

3.2 Choice of Snapshot Density

Since Δt depends on N_{snap}, increasing N_{snap} refines the integration grid. To justify the choice $N_{\text{snap}} = 60$, a refinement study was performed for the circular test case using the Euler–Cromer method, with N_{between} held fixed. Numerical accuracy was quantified via the relative energy variation

$$\delta_E = \frac{E_{\max} - E_{\min}}{|E_0|}.$$

Table 1: Energy variation under timestep refinement for Euler–Cromer (circular orbit).

N_{snap}	Δt (yr)	δ_E
30	1.67×10^{-3}	1.10×10^{-4}
60	8.33×10^{-4}	2.74×10^{-5}
120	4.17×10^{-4}	6.85×10^{-6}

Doubling N_{snap} (halving Δt) reduces δ_E by approximately a factor of four. Although Euler–Cromer is formally first order in Δt, the bounded oscillatory nature of the energy error in

conservative systems leads to approximate quadratic scaling of the error. Increasing N_{snap} beyond 60 further reduces error but with diminishing absolute improvement relative to computational cost. Accordingly, $N_{\text{snap}} = 60$ was adopted as a balanced choice between numerical accuracy and efficiency.

To assess stability more broadly, simulations were also performed with Δt increased by a factor of ten on the circular and elliptical cases.

3.3 Integration Schemes

The Euler–Cromer scheme updates velocity before position:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}(\mathbf{r}_n)\Delta t, \tag{4}$$

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1}\Delta t. \tag{5}$$

The second-order Runge–Kutta (midpoint) method advances the full state $\mathbf{y} = (x, y, v_x, v_y)$ by evaluating the acceleration at an intermediate time. Defining the system in first-order form $\dot{\mathbf{y}} = f(\mathbf{y})$, the scheme proceeds as

$$\mathbf{k}_1 = f(\mathbf{y}_n), \tag{6}$$

$$\mathbf{y}_{n+\frac{1}{2}} = \mathbf{y}_n + \frac{\Delta t}{2}\mathbf{k}_1, \tag{7}$$

$$\mathbf{k}_2 = f(\mathbf{y}_{n+\frac{1}{2}}), \tag{8}$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \Delta t\, \mathbf{k}_2. \tag{9}$$

3.4 Double-Star Gravitational Field

To investigate the effect of a non-central potential, the single central mass was replaced by two fixed gravitating centres separated by 0.4 AU. The total gravitational parameter was preserved by assigning

$$GM_1 = GM_2 = \frac{GM}{2},$$

with the stars located at $(-0.2,\, 0)$ and $(+0.2,\, 0)$ in the orbital plane.

The acceleration on the test particle is therefore the vector sum of the two inverse-square contributions,

$$\mathbf{a}(\mathbf{r}) = -\frac{GM_1}{r_1^3}(\mathbf{r} - \mathbf{r}_1) - \frac{GM_2}{r_2^3}(\mathbf{r} - \mathbf{r}_2), \tag{10}$$

where $r_i = |\mathbf{r} - \mathbf{r}_i|$ denotes the distance to each star.

In the numerical implementation, the acceleration was computed at each timestep by evaluating the displacement vectors to both centres and summing their contributions before advancing the state using the Euler–Cromer method .

3.5 Sun–Jupiter Perturbation Model (Encke-like Orbit)

To investigate gravitational perturbations in a physically motivated setting, the model was extended to include Jupiter as a secondary gravitating body. The Sun was fixed at the origin with $GM_\odot = 4\pi^2$, and Jupiter was treated as a point mass moving on a circular orbit of radius $a_J = 5.2$ AU with angular frequency

$$\omega_J = \frac{2\pi}{T_J}, \qquad T_J = a_J^{3/2}.$$

Jupiter's gravitational parameter was set to

$$GM_J = 9.543 \times 10^{-4}\, GM_\odot,$$

consistent with the known mass ratio.

At time t, Jupiter's position is therefore

$$\mathbf{r}_J(t) = \big(a_J \cos(\omega_J t),\ a_J \sin(\omega_J t)\big).$$

The test particle experiences the combined acceleration

$$\mathbf{a}(\mathbf{r}, t) = -\frac{GM_\odot}{r^3}\mathbf{r} - \frac{GM_J}{|\mathbf{r} - \mathbf{r}_J(t)|^3}\left(\mathbf{r} - \mathbf{r}_J(t)\right). \tag{11}$$

This introduces a time-dependent, non-central perturbation to the otherwise Keplerian motion.

To model a short-period comet largely controlled by the Sun, an Encke-like orbit was constructed using approximate orbital elements $a \approx 2.218\,\mathrm{AU}$ and perihelion distance $q \approx 0.339\,\mathrm{AU}$. The initial condition was set at perihelion with tangential speed determined from the vis-viva equation,

$$v_p = \sqrt{GM_\odot \left(\frac{2}{q} - \frac{1}{a}\right)}. \tag{12}$$

The timestep was defined using the same snapshot scheme, with the orbital period given by $T = a^{3/2} \approx 3.30\,\mathrm{yr}$.

Numerically, Jupiter's position was updated at every integration step using the accumulated simulation time t, and its gravitational contribution was added to the Sun's acceleration before advancing the state vector. The integrator itself was unchanged from the two-body Euler-Cromer case.

4 Results

4.1 Exercise 1: Two–Body Orbits (Euler–Cromer)

All simulations used $N_{\mathrm{snap}} = 60$ and $N_{\mathrm{between}} = 20$, giving

$$\Delta t = 8.33 \times 10^{-4}\ \mathrm{yr}.$$

(a) Circular Initial Conditions

The orbit remains bounded and nearly circular. Radial variation is small,

$$r_{\min} = 0.9974\ \mathrm{AU}, \qquad r_{\max} = 1.0026\ \mathrm{AU},$$

with

$$\frac{r_{\max}}{r_{\min}} = 1.00525.$$

The relative energy variation is

$$\frac{E_{\max} - E_{\min}}{|E_0|} = 2.74 \times 10^{-5},$$

while angular momentum is conserved to 8.76×10^{-15}. The motion is stable and periodic over 8 orbits.

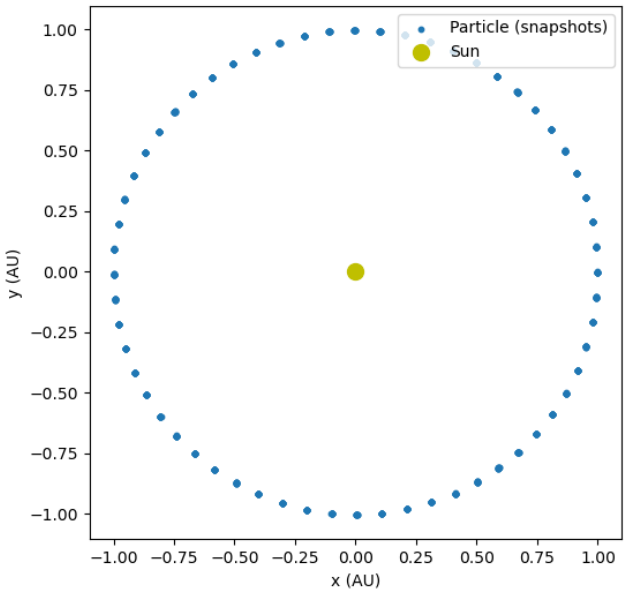


Figure 1: Circular orbit computed using Euler–Cromer.

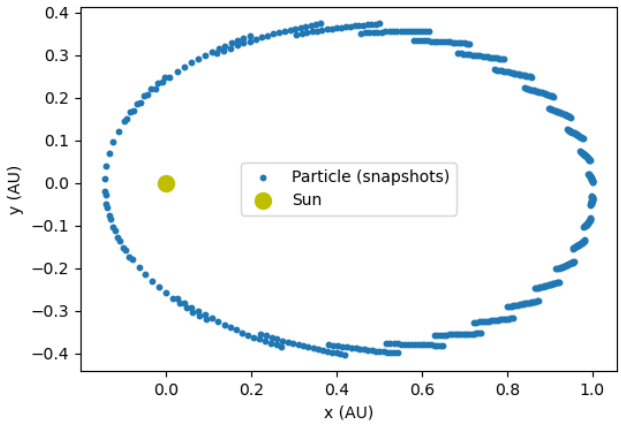


Figure 2: Eccentric orbit computed using Euler–Cromer.

(b) Elliptical Initial Conditions

The orbit is strongly eccentric, with

$$r_{\min} = 0.1431 \text{ AU}, \qquad r_{\max} = 1.0000 \text{ AU},$$

and

$$\frac{r_{\max}}{r_{\min}} = 6.99.$$

Energy variation increases to

$$\frac{E_{\max} - E_{\min}}{|E_0|} = 0.241,$$

while angular momentum remains conserved to machine precision. The trajectory remains bounded but exhibits visible phase distortion across successive orbits.

Euler–Cromer therefore preserves angular momentum accurately but shows substantial energy oscillation for eccentric motion.

4.2 Exercise 2: Two–Body Orbits (RK2)

The timestep was unchanged.

(a) Circular Initial Conditions

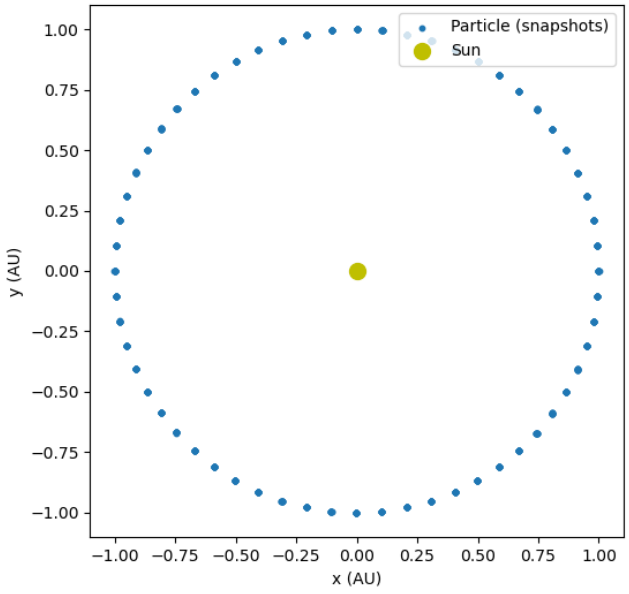


Figure 3: Circular orbit computed using RK2.

Radial variation is negligible:

$$r_{\min} = 1.00000055 \text{ AU}, \qquad r_{\max} = 1.00002141 \text{ AU},$$

giving

$$\frac{r_{\max}}{r_{\min}} = 1.00002135.$$

Energy variation reduces to

$$8.99 \times 10^{-7},$$

with angular momentum variation 4.50×10^{-7}. The orbit remains closed and highly stable.

(b) Elliptical Initial Conditions

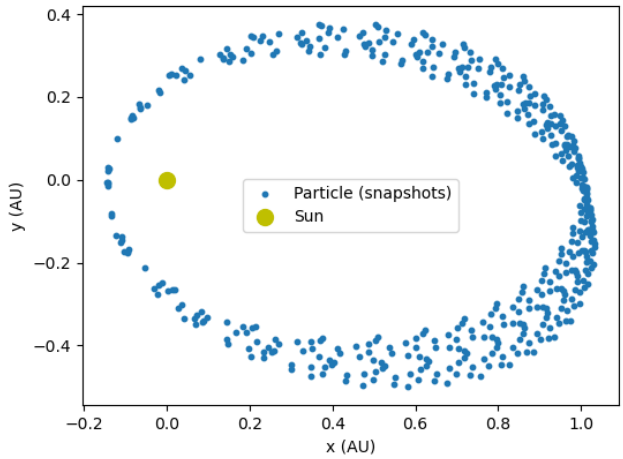


Figure 4: Eccentric orbit computed using RK2.

$$r_{\min} = 0.1430 \text{ AU}, \qquad r_{\max} = 1.0497 \text{ AU}, \qquad \frac{r_{\max}}{r_{\min}} = 7.34.$$

Energy variation is

$$4.86 \times 10^{-2},$$

and angular momentum variation

$$3.61 \times 10^{-3}.$$

The orbit remains bounded but does not retrace a single closed ellipse.

At identical timestep, RK2 reduces invariant variation by roughly two orders of magnitude for circular motion and substantially improves eccentric behaviour relative to Euler–Cromer.

4.3 Exercise 3: Increased Timestep ($\Delta t \times 10$)

The timestep was increased by a factor of 10 to

$$\Delta t = 8.33 \times 10^{-3} \text{ yr}.$$

(a) Circular Initial Conditions

$$r_{\min} = 0.9752 \text{ AU}, \qquad r_{\max} = 1.0276 \text{ AU}, \qquad \frac{r_{\max}}{r_{\min}} = 1.0537.$$

Energy variation rises to

$$2.74 \times 10^{-3},$$

while angular momentum remains conserved. The orbit remains bounded but visibly thickened.

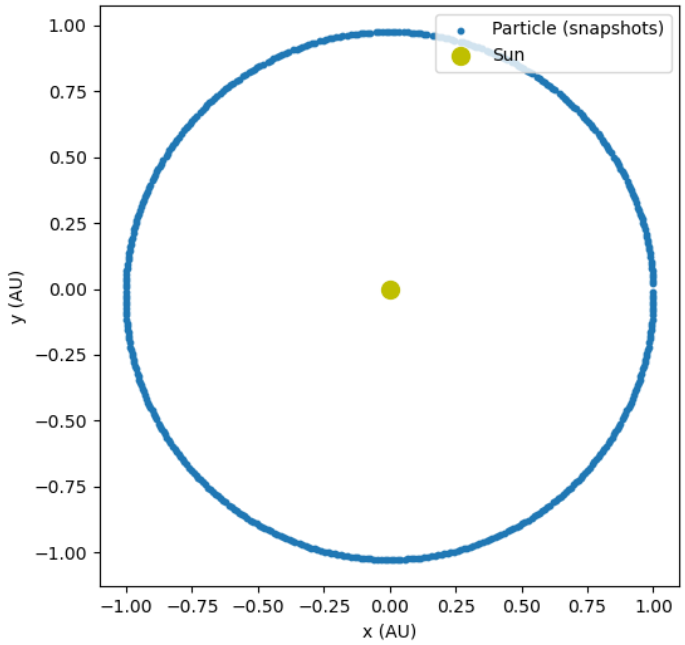


Figure 5: Circular orbit with increased timestep.

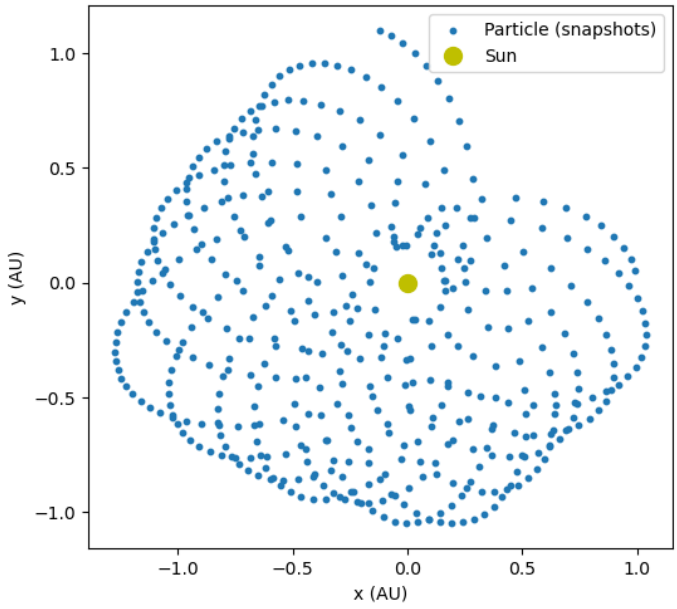


Figure 6: Eccentric orbit with increased timestep.

(b) Elliptical Initial Conditions

$$r_{\min} = 0.1602 \text{ AU}, \qquad r_{\max} = 1.3175 \text{ AU}, \qquad \frac{r_{\max}}{r_{\min}} = 8.22.$$

Energy variation increases to

$$2.29,$$

indicating severe distortion. The trajectory expands and no longer approximates a Keplerian orbit.

Accuracy degrades markedly once the perihelion timescale is under-resolved.

4.4 Exercise 4: Double–Star Potential

The two stars were fixed at $(+0.2, 0)$, and $(-0.2, 0)$ AU with equal mass. The timestep was unchanged.

(a) Circular Initial Conditions

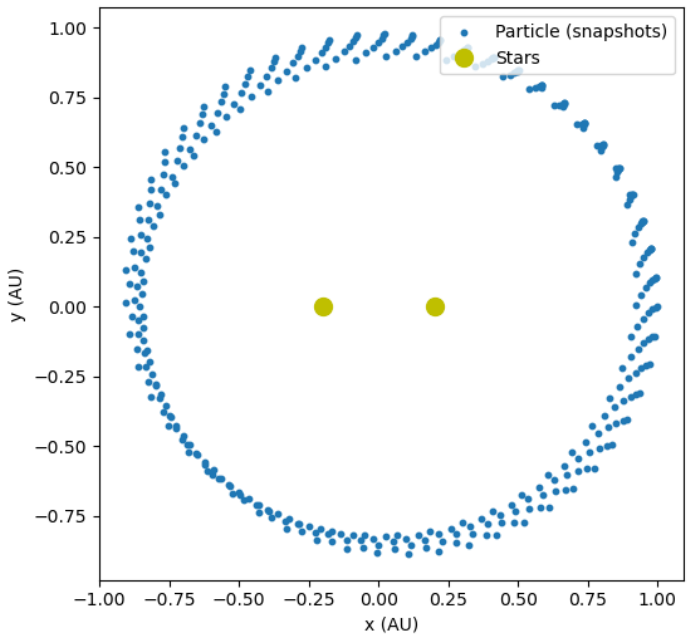


Figure 7: Motion in the double-star potential (circular initial conditions).

$$r_{\min} = 0.8223 \text{ AU}, \qquad r_{\max} = 0.9995 \text{ AU}, \qquad \frac{r_{\max}}{r_{\min}} = 1.215.$$

The orbit remains bounded but non-periodic and does not close.

(b) Reduced Tangential Velocity

$$r_{\min} = 0.1535 \text{ AU}, \qquad r_{\max} = 414.7 \text{ AU}.$$

The particle escapes, and the trajectory diverges rapidly.

The two-centre field produces bounded but non-closed motion for near-circular conditions and unbounded escape for reduced tangential velocity, demonstrating strong sensitivity to initial conditions.

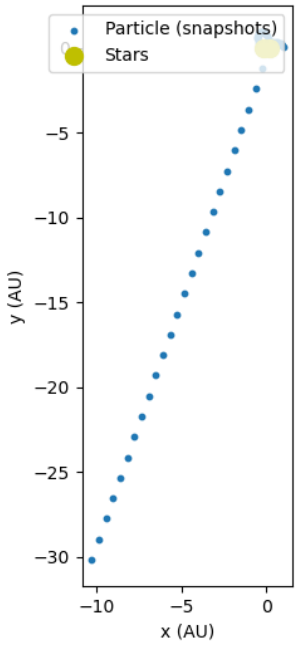


Figure 8: Motion in the double-star potential (reduced tangential velocity).

4.5 Extension: Encke-like Orbit With and Without Jupiter

An Encke-like orbit ($a \approx 2.218$ AU, $q \approx 0.339$ AU) was integrated using a fixed timestep derived from same conditions as 1a) but with a period of 3.3 years this gave a timstep of:

$$\Delta t = 2.75 \times 10^{-3} \text{ yr},$$

for 10, 20, and 30 orbital periods. Two models were compared: (i) Sun-only (two-body), and (ii) Sun+Jupiter, with Jupiter on a circular orbit at $a_J = 5.2$ AU. For each run, the radial extrema and the relative variations of specific energy and angular momentum were recorded.

Quantitative comparison

To isolate the effect of Jupiter, the relative variations in specific energy and angular momentum were recorded over 10, 20, and 30 orbital periods. The timestep was identical in all runs, $\Delta t = 2.75 \times 10^{-3}$ yr.

Table 2: Relative variation of invariants for the Encke-like orbit.

Model	Orbits	$\Delta E/\lvert E_0\rvert$	$\Delta h/\lvert h_0\rvert$
Sun only	10	0.4477	8.57×10^{-15}
Sun only	20	0.4480	1.93×10^{-14}
Sun only	30	0.4480	1.93×10^{-14}
Sun+Jupiter	10	0.4674	0.0138
Sun+Jupiter	20	0.4910	0.0226
Sun+Jupiter	30	0.5079	0.0322

In the Sun-only model, angular momentum is conserved to machine precision ($\sim 10^{-14}$) across all integration lengths, and the energy variation remains essentially constant as the number of orbits

increases. This indicates bounded numerical error without secular growth and confirms that the motion remains effectively integrable and repeatable over 30 periods.

With Jupiter included, angular momentum about the origin varies at the percent level, reflecting the presence of a non-central, time-dependent perturbation. The relative energy variation increases modestly with integration length, indicating accumulated perturbative and truncation effects over long timescales. Despite this, the motion remains spatially bounded over 30 periods.

The contrast between the two models demonstrates the transition from integrable two-body motion, characterised by exact conservation of angular momentum and repeatable trajectories, to perturbed three-body dynamics, where torque from the outer planet introduces secular evolution and loss of periodic closure.

This shows that it is incredibly difficult to solidly model a real object with a low integrate method due to the many different pertubing forces.

20-orbit trajectories (visual comparison)

For clarity, Fig. 9 and Fig. 10 show only the 20-orbit integrations.

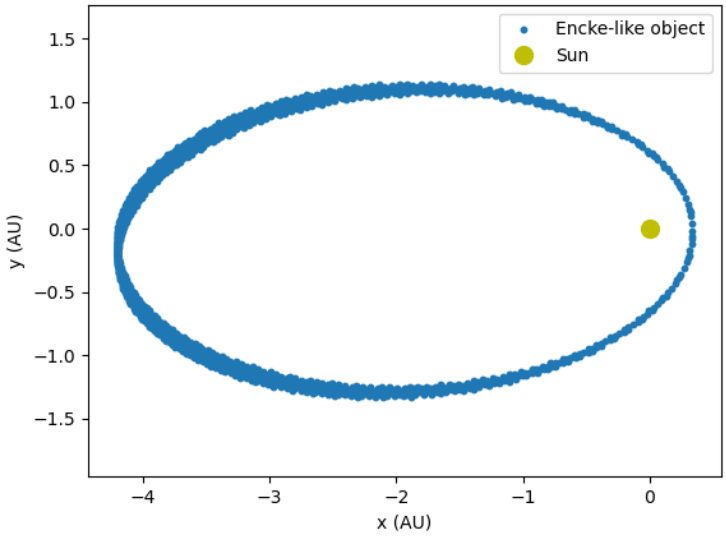


Figure 9: Sun-only Encke-like orbit integrated over 20 periods. The trajectory remains bounded and nearly repeatable.

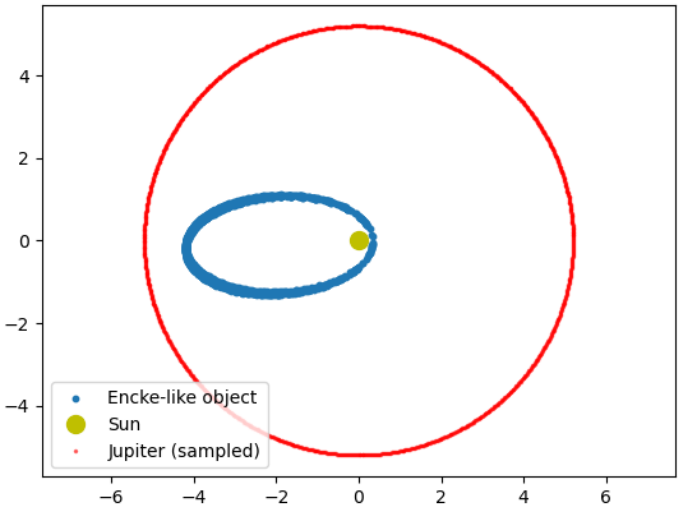


Figure 10: Encke-like orbit integrated over 20 periods with Jupiter included. The trajectory shows clear non-closure and gradual thickening consistent with perturbative evolution.

5 Discussion

The results establish a clear hierarchy of numerical performance and physical behaviour.

For the integrable two-body problem, Euler–Cromer produces bounded energy oscillations and exact angular-momentum conservation, but exhibits noticeable phase distortion in eccentric motion. The RK2 scheme significantly reduces invariant variation at identical timestep, confirming the expected improvement associated with second-order truncation error. Timestep refinement demonstrates numerical convergence, while timestep enlargement reveals the breakdown of fidelity once the perihelion timescale is insufficiently resolved.

Replacing the central potential with a two-centre field removes rotational symmetry and hence conservation of angular momentum about the origin. The resulting non-closure and sensitivity to initial conditions are physical consequences of broken integrability rather than numerical artefacts.

The Sun–Jupiter extension further illustrates this transition. In the Sun-only model, angular momentum remains conserved to machine precision and the orbit is effectively repeatable over tens of periods. Inclusion of Jupiter introduces a time-dependent torque, leading to measurable angular-momentum variation and secular evolution of the trajectory, while remaining spatially bounded over the simulated timescale.

Collectively, the simulations distinguish clearly between numerical error, timestep sensitivity, and genuine dynamical complexity introduced by multi-body perturbations.

6 Conclusion

This investigation demonstrates that low-order numerical schemes can accurately reproduce Keplerian motion provided the timestep resolves the shortest dynamical timescale. The RK2 method offers substantially improved invariant conservation relative to Euler–Cromer at fixed timestep, and convergence must be verified explicitly through systematic refinement.

Increasing the timestep leads to predictable degradation consistent with truncation-error theory, particularly for highly eccentric orbits. Extending the model to multi-body configurations reveals qualitative dynamical transitions: loss of rotational symmetry, non-conservation of angular momentum about the origin, and secular orbital evolution under perturbation.

The study therefore illustrates both the mathematical structure of integrable gravitational sys-

tems and the numerical requirements necessary to model their perturbations reliably.

References

[1] D. J. Griffiths, *Introduction to Classical Mechanics: With Problems and Solutions*, Cambridge University Press, 2017.

[2] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd ed., Springer, 2006.

[3] C. D. Murray and S. F. Dermott, *Solar System Dynamics*, Cambridge University Press, 1999.

A Numerical Implementation Code

The complete Python source code used for all simulations in this report is provided in the following appendix. The code includes implementations of the Euler–Cromer and RK2 integrators, the double–star model, and the Sun–Jupiter perturbation extension.

In [56]:
```
import numpy as np
import matplotlib.pyplot as plt
#define GM
GM = 4 * np.pi**2
```

Excersize 1A)

In [79]:
```
# reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

n_orbits = 8
n_snapshots_per_orbit = 60
steps_between_snapshots = 20

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt definition
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage
xs, ys = [], []
vxs, vys = [], []

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3

        # Euler-Cromer update
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
```

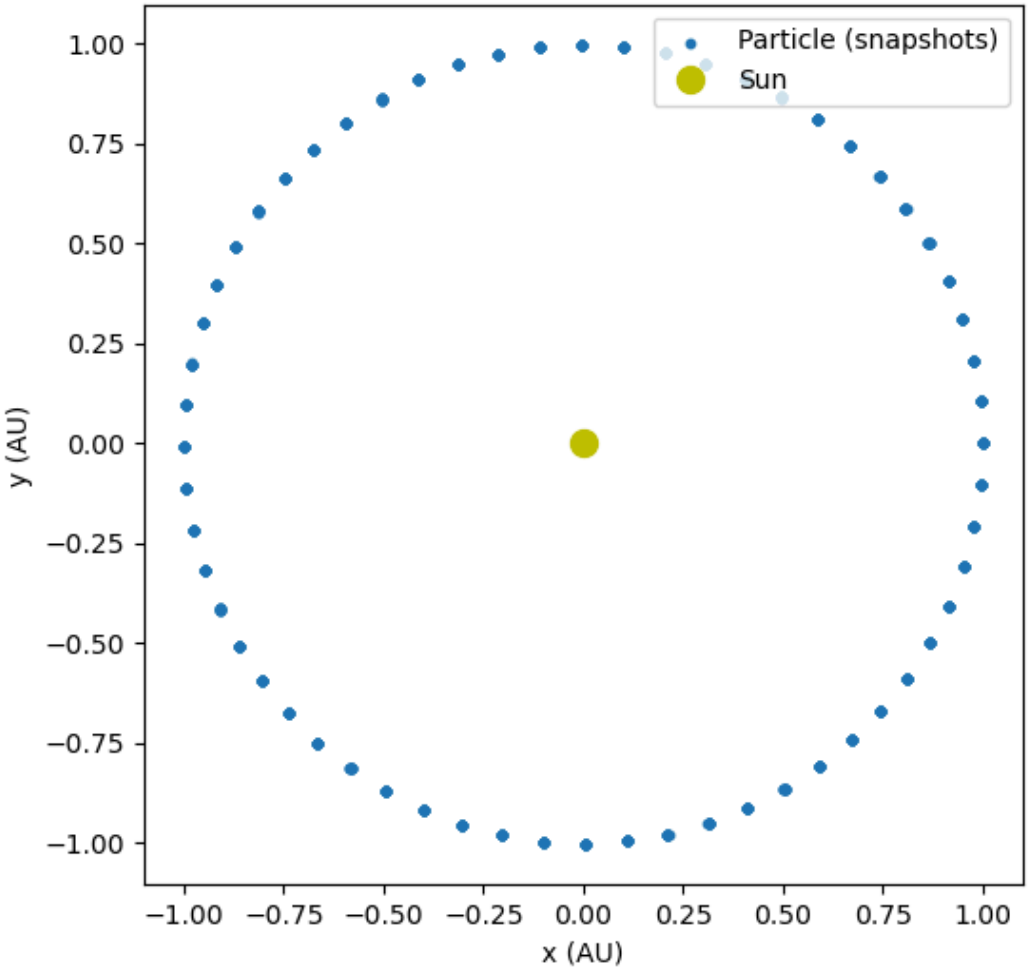
```
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('1a')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 0.9973957746869052 1.0026316423229507
r ratio = 1.005249538617395
Energy drift (max-min)/|E0| = 2.741291912140694e-05
Angular momentum drift (max-min)/|h0| = 8.764195122255021e-15
```



1B)

In [80]:
```
# reset for B
x, y = 1.0, 0.0
vx, vy = 0.0, np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

n_orbits = 8
n_snapshots_per_orbit = 60
steps_between_snapshots = 20

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt definition
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage
xs, ys = [], []
vxs, vys = [], []

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3

        # Euler-Cromer update
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

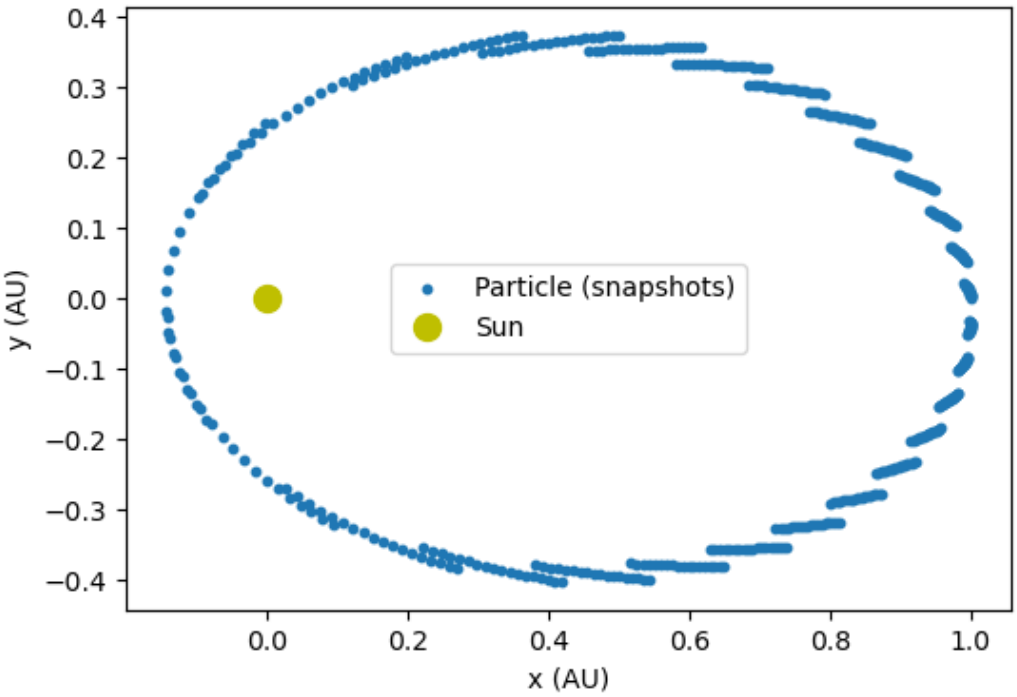
# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
```

```
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend()
plt.savefig('1b')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 0.14308180885035301 1.000001434863174
r ratio = 6.989018680278634
Energy drift (max-min)/|E0| = 0.2410421548780837
Angular momentum drift (max-min)/|h0| = 5.0888874903416275e-15
```



Excersize 2 A) (RK2)

In [81]:
```
#reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
n_orbits = 8
```

```
n_snapshots_per_orbit = 60        # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):

        # RK2 (midpoint) update for state [x, y, vx, vy]

        # k1 from current state
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3

        k1x, k1y = vx, vy
        k1vx, k1vy = ax, ay

        # midpoint state
        x_mid  = x  + 0.5 * dt * k1x
        y_mid  = y  + 0.5 * dt * k1y
        vx_mid = vx + 0.5 * dt * k1vx
        vy_mid = vy + 0.5 * dt * k1vy

        # k2 from midpoint state
        r_mid = np.hypot(x_mid, y_mid)
        ax_mid = -GM * x_mid / r_mid**3
        ay_mid = -GM * y_mid / r_mid**3

        k2x, k2y = vx_mid, vy_mid
        k2vx, k2vy = ax_mid, ay_mid

        # advance full step using k2
        x  += dt * k2x
        y  += dt * k2y
        vx += dt * k2vx
        vy += dt * k2vy

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

xs = np.array(xs); ys = np.array(ys)
vxs = np.array(vxs); vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
```

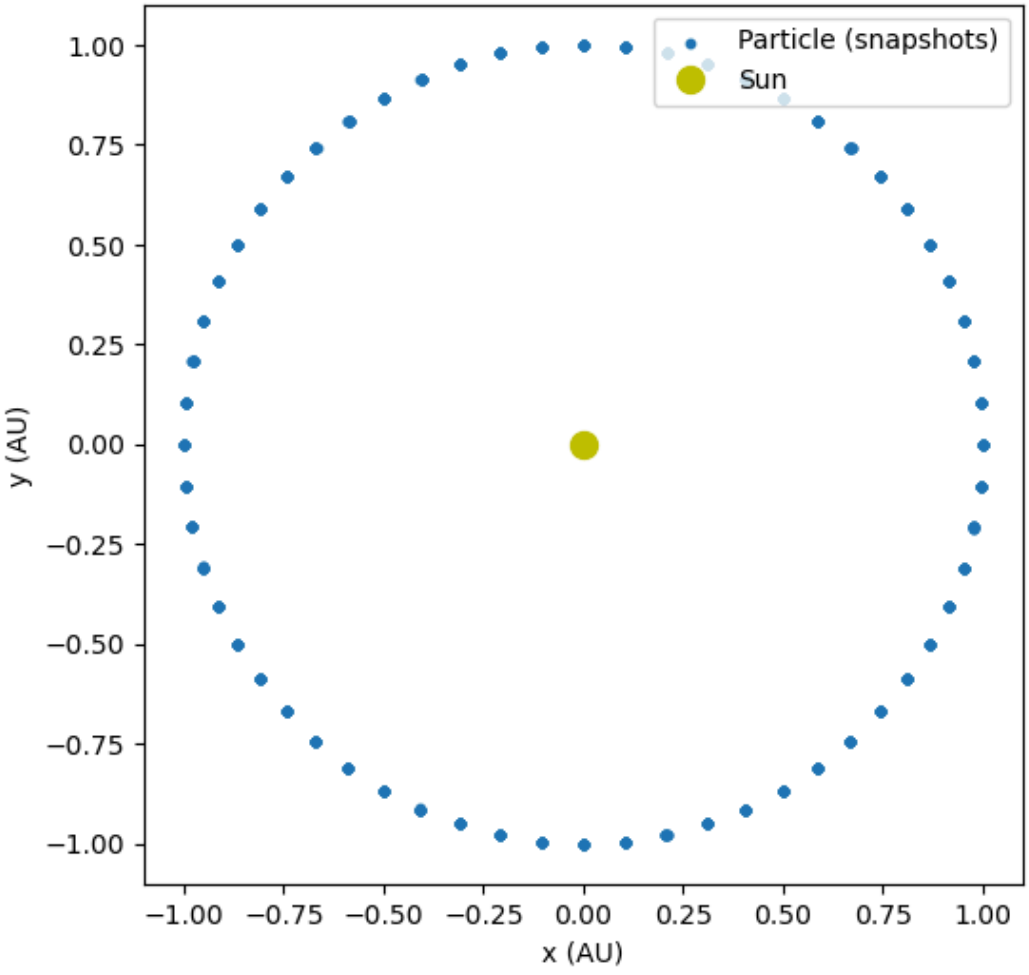
```
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = (vxs**2 + vys**2)
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc='upper right')
plt.savefig('2a')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 1.0000000553840578 1.0000214059197579
r ratio = 1.0000213505345177
Energy drift (max-min)/|E0| = 8.999578870102564e-07
Angular momentum drift (max-min)/|h0| = 4.499798260385103e-07
```



2B)

In [82]:
```
#reset
x, y = 1.0, 0.0
vx, vy = 0.0,  np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
n_orbits = 8
n_snapshots_per_orbit = 60        # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):

        # RK2 (midpoint) update for state [x, y, vx, vy]

        # k1 from current state
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3

        k1x, k1y = vx, vy
        k1vx, k1vy = ax, ay

        # midpoint state
        x_mid  = x  + 0.5 * dt * k1x
        y_mid  = y  + 0.5 * dt * k1y
        vx_mid = vx + 0.5 * dt * k1vx
        vy_mid = vy + 0.5 * dt * k1vy

        # k2 from midpoint state
        r_mid = np.hypot(x_mid, y_mid)
        ax_mid = -GM * x_mid / r_mid**3
        ay_mid = -GM * y_mid / r_mid**3

        k2x, k2y = vx_mid, vy_mid
        k2vx, k2vy = ax_mid, ay_mid

        # advance full step using k2
        x  += dt * k2x
        y  += dt * k2y
        vx += dt * k2vx
```

```
        vy += dt * k2vy

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

xs = np.array(xs); ys = np.array(ys)
vxs = np.array(vxs); vys = np.array(vys)

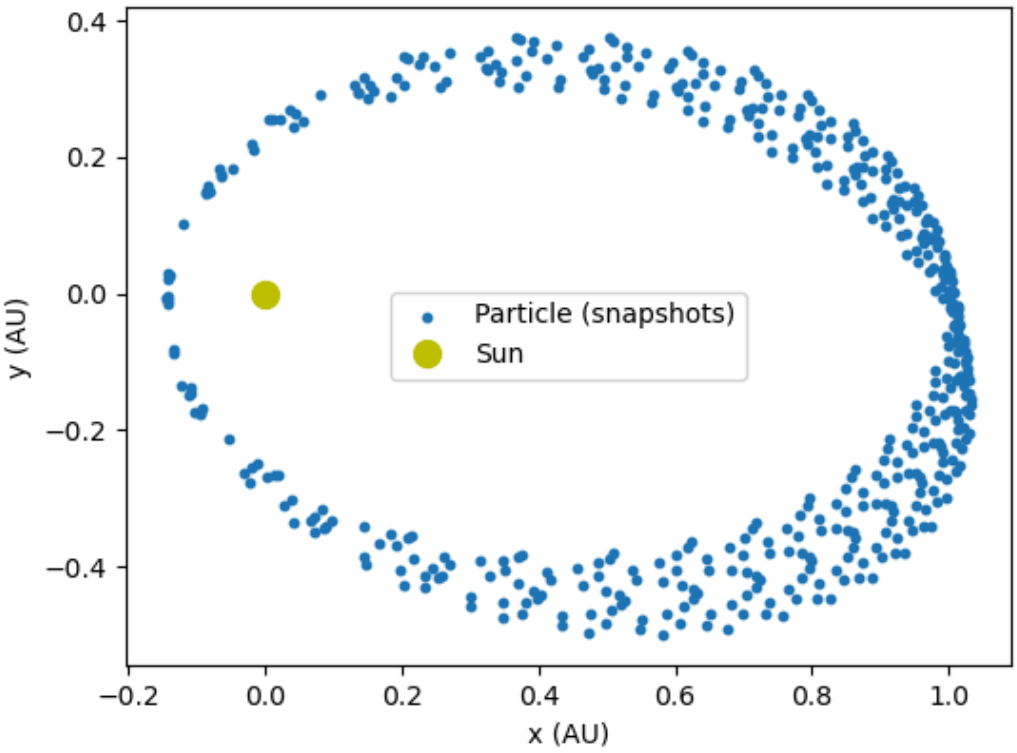
# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = (vxs**2 + vys**2)
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend()
plt.savefig('2b')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 0.1430176463618069 1.0497035003500041
r ratio = 7.339678193937397
Energy drift (max-min)/|E0| = 0.04864961786196357
Angular momentum drift (max-min)/|h0| = 0.0036112002441622327
```



3A)

Timestep increases by 10 by decreasing number of steps between snapshots from 20 to 2 initial conditians from 1a

In [83]:
```python
# reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

n_orbits = 8
n_snapshots_per_orbit = 60
steps_between_snapshots = 2

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt definition
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage
xs, ys = [], []
vxs, vys = [], []

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
```

```
        # Euler-Cromer update
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

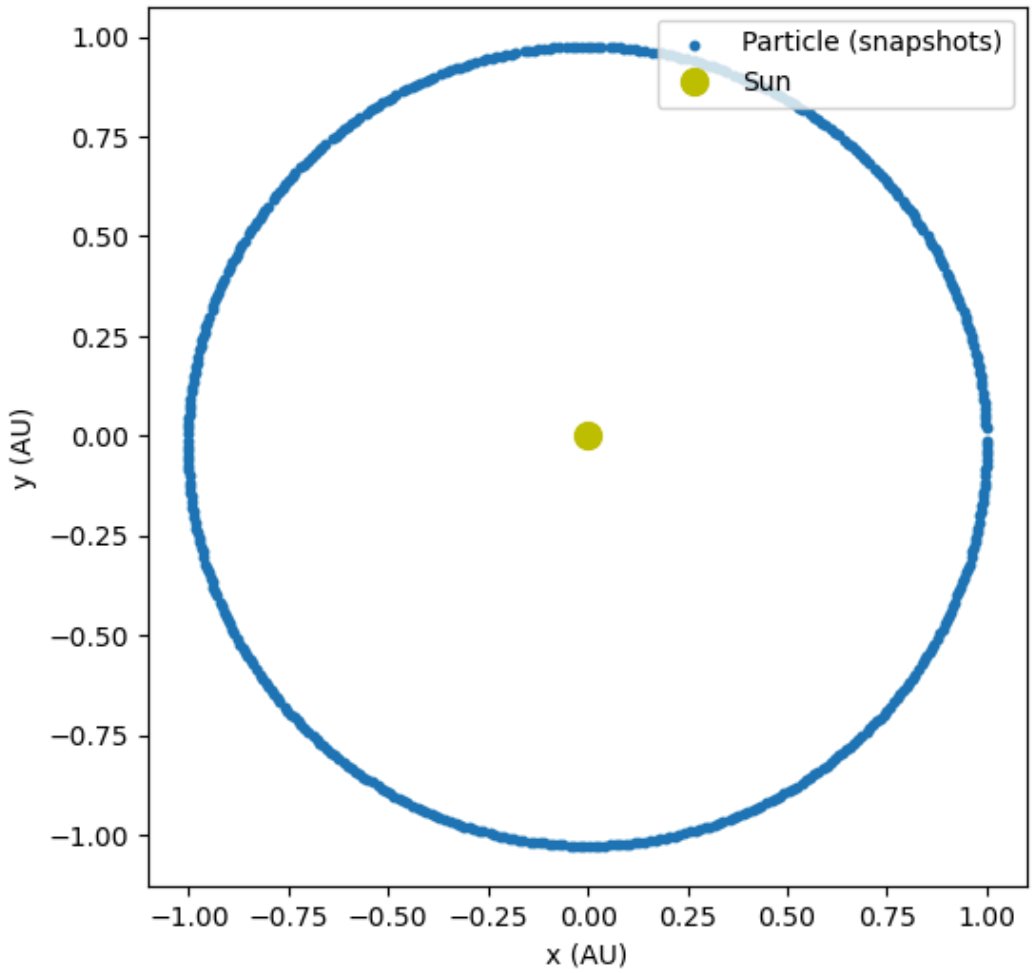
# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('3a')
plt.show()
```

```
Timestep = 0.008333333333333333
r min/max = 0.97518791172633 1.0275558471867166
r ratio = 1.0537003533684928
Energy drift (max-min)/|E0| = 0.002743376226745127
Angular momentum drift (max-min)/|h0| = 2.261727773485168e-15
```



3B)

Timestep increases by 10 by decreasing number of steps between snapshots from 20 to 2 initial conditians from 1b

In [84]:
```
# reset
x, y = 1.0, 0.0
vx, vy = 0.0,  np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

n_orbits = 8
n_snapshots_per_orbit = 60
steps_between_snapshots = 2

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt definition
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage
xs, ys = [], []
```

```
vxs, vys = [], []

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3

        # Euler-Cromer update
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

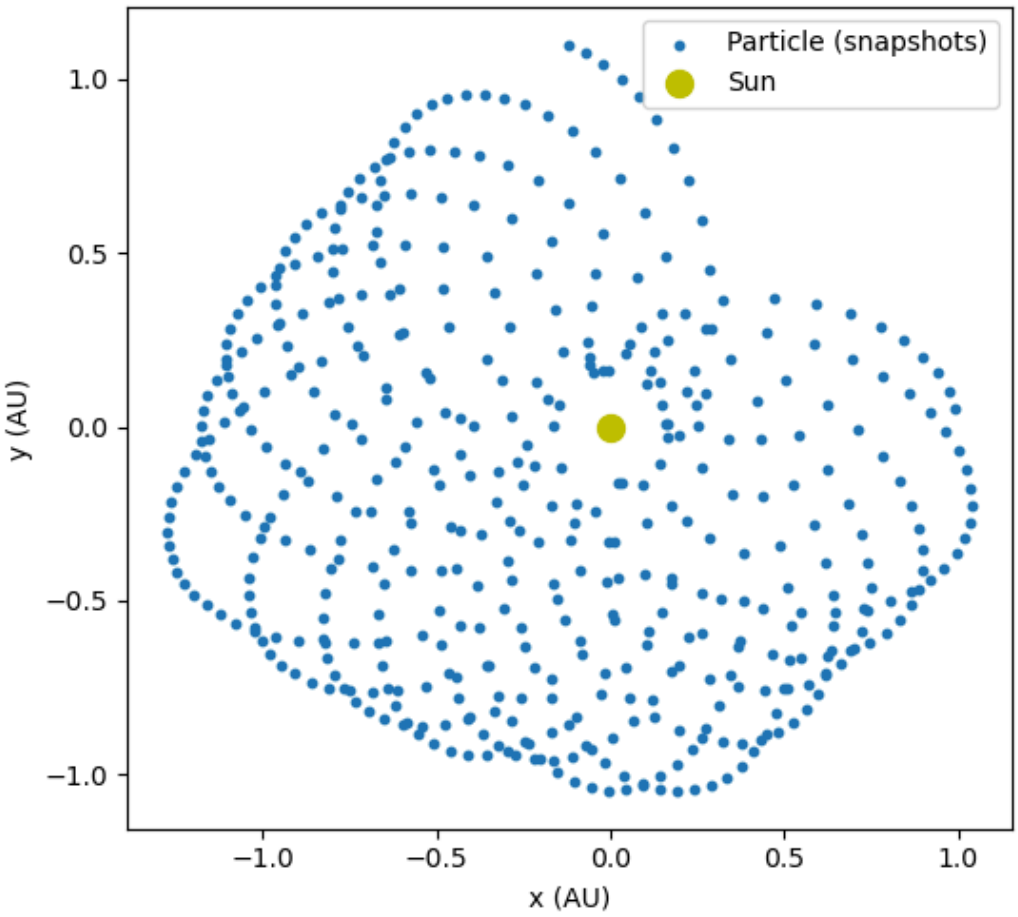
# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('3b')
plt.show()
```

```
Timestep = 0.008333333333333333
r min/max = 0.1602489127428004 1.3175339496115512
r ratio = 8.221796498090406
Energy drift (max-min)/|E0| = 2.288874955426488
Angular momentum drift (max-min)/|h0| = 2.4030857593279904e-15
```



4A)

Double star seperated by 0.4 AU (Assumption is one on x = +0.2 AU and one on x = -0.2 AU)

Initial Cndts 1a)

In [85]:
```
# reset
x, y = 1, 0.0
vx, vy = 0.0, 2 * np.pi

# ---- double-star parameters ----
d = 0.4
x1, y1 = -d/2, 0.0    # star 1 at (-0.2, 0)
x2, y2 =  d/2, 0.0    # star 2 at (+0.2, 0)

GM_total = GM          # keep total mass same as single-sun case
GM1 = GM_total / 2
GM2 = GM_total / 2

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
```

```
n_orbits = 6
n_snapshots_per_orbit = 60        # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs,vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):

        # distances to each star
        dx1, dy1 = x - x1, y - y1
        dx2, dy2 = x - x2, y - y2
        r1 = np.hypot(dx1, dy1)
        r2 = np.hypot(dx2, dy2)

        # acceleration = sum of the two gravitational accelerations
        ax = -GM1 * dx1 / r1**3 - GM2 * dx2 / r2**3
        ay = -GM1 * dy1 / r1**3 - GM2 * dy2 / r2**3

        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)


# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)


# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

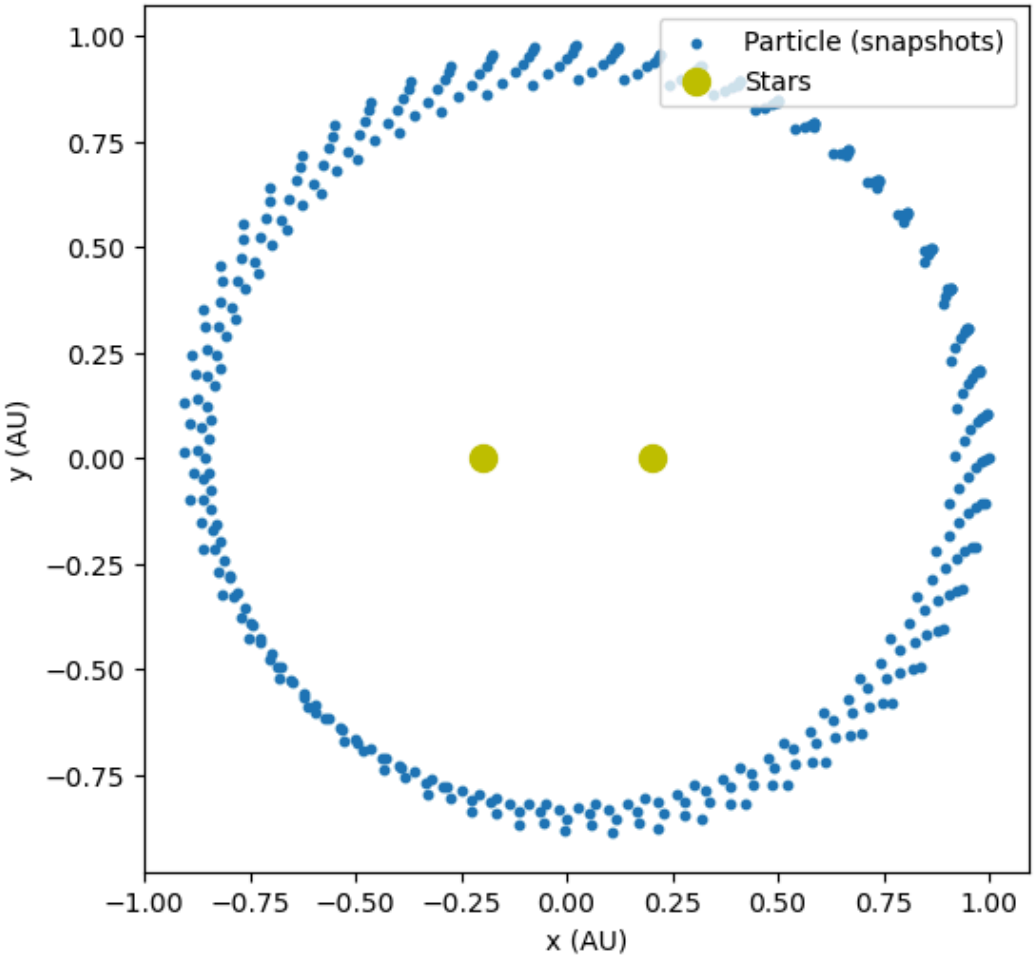
# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
```

```
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot([x1, x2], [y1, y2], 'yo', markersize=10, label="Stars")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('4a')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 0.8223108234054474 0.9994695271843844
r ratio = 1.2154400729462216
Energy drift (max-min)/|E0| = 0.20735739798375832
Angular momentum drift (max-min)/|h0| = 0.07773178608444327
```



4B)

In [86]:
```
# reset
x, y = 1, 0.0
vx, vy = 0.0, np.pi

# ---- double-star parameters ----
```

```
d = 0.4
x1, y1 = -d/2, 0.0    # star 1 at (-0.2, 0)
x2, y2 =  d/2, 0.0    # star 2 at (+0.2, 0)

GM_total = GM          # keep total mass same as single-sun case
GM1 = GM_total / 2
GM2 = GM_total / 2

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
n_orbits = 1
n_snapshots_per_orbit = 60        # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs,vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):

        # distances to each star
        dx1, dy1 = x - x1, y - y1
        dx2, dy2 = x - x2, y - y2
        r1 = np.hypot(dx1, dy1)
        r2 = np.hypot(dx2, dy2)

        # acceleration = sum of the two gravitational accelerations
        ax = -GM1 * dx1 / r1**3 - GM2 * dx2 / r2**3
        ay = -GM1 * dy1 / r1**3 - GM2 * dy2 / r2**3

        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)


# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)
```

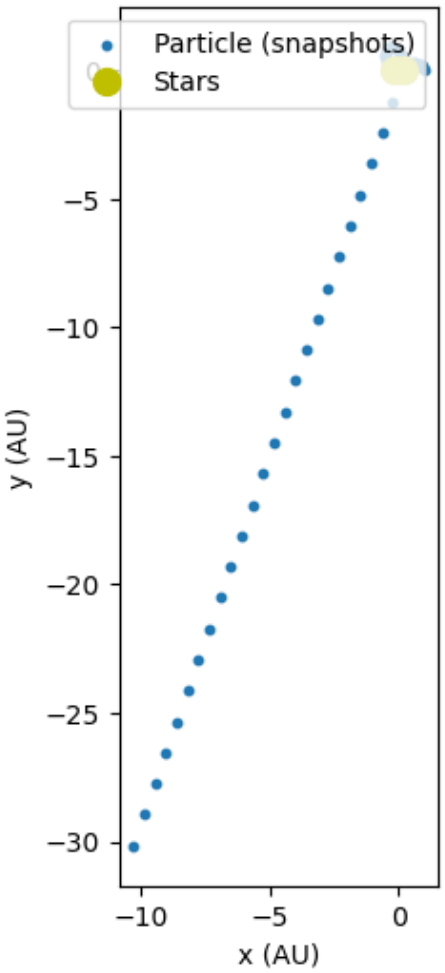
```
# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)

# orbit plot
plt.figure(figsize=(6,6))
plt.scatter(xs, ys, s=10, label="Particle (snapshots)")
plt.plot([x1, x2], [y1, y2], 'yo', markersize=10, label="Stars")
plt.gca().set_aspect("equal", adjustable="box")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('4b')
plt.show()
```

```
Timestep = 0.0008333333333333334
r min/max = 0.15345327210300733 31.867318754878337
r ratio = 207.66789992908735
Energy drift (max-min)/|E0| = 87.61290083154539
Angular momentum drift (max-min)/|h0| = 5.798221731152496
```



5 Extension- First Modelling Encke around Sun

In [87]:
```
a_encke = 2.218   # AU
q_encke = 0.339   # AU

x, y = q_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/q_encke - 1.0/a_encke))  # perihelion speed (AU/
dt = 0.001
n_steps = 5000

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = 3.3

# You choose:
n_orbits = 20
n_snapshots_per_orbit = 60     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)
```

```
# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

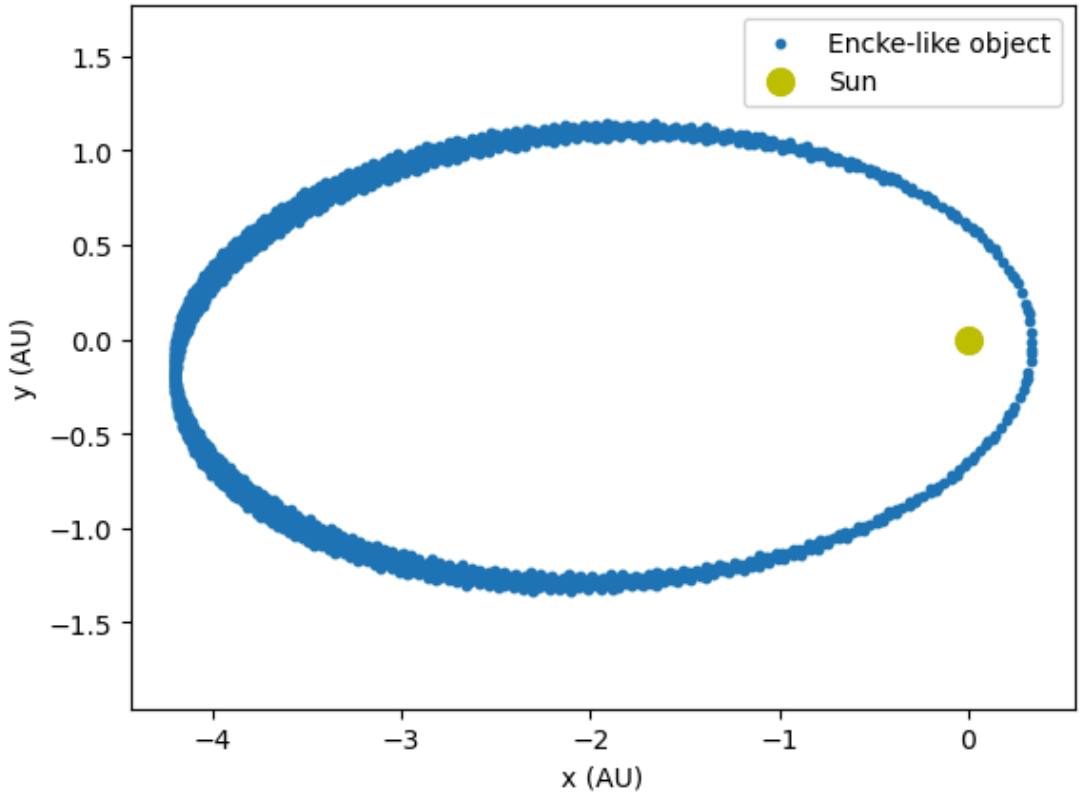
plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.axis("equal")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('5a')
plt.show()

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```




```
Timestep = 0.00275
r min/max = 0.33901405432311427 4.205819463961515
r ratio = 12.406032759789213
Energy drift (max-min)/|E0| = 0.4480005599793379
Angular momentum drift (max-min)/|h0| = 1.9292657027336464e-14
```

In [88]:

```python
import numpy as np
import matplotlib.pyplot as plt


# Encke-like initial conditions (planar approximation)
# Use orbital elements (approx): a = 2.218 AU, q = 0.339 AU (so e ~ 0.847)
# Start at perihelion on +x axis, velocity purely +y (tangential).

a_encke = 2.218   # AU
p_encke = 0.339   # AU

x, y = p_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/p_encke - 1.0/a_encke))  # perihelion speed (AU/

dt = 0.001
n_steps = 5000

# ----------------------------
# Jupiter parameters (2D circular orbit approximation)
# ----------------------------
aJ = 5.2
T_J = aJ**1.5
omega_J = 2*np.pi / T_J

GM_sun = GM
GM_jup = 0.0009543 * GM_sun

# period from Kepler
```

```
T = 3.3

# You choose:
n_orbits = 20
n_snapshots_per_orbit = 60
steps_between_snapshots = 20

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []
xJ_list, yJ_list = [], []

t = 0.0

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):

        # ---- Sun acceleration ----
        r = np.hypot(x, y)
        ax_sun = -GM_sun * x / r**3
        ay_sun = -GM_sun * y / r**3

        # ---- Jupiter position on circular orbit ----
        xJ = aJ * np.cos(omega_J * t)
        yJ = aJ * np.sin(omega_J * t)

        # ---- Jupiter acceleration ----
        dxJ, dyJ = x - xJ, y - yJ
        rJ = np.hypot(dxJ, dyJ)
        ax_jup = -GM_jup * dxJ / rJ**3
        ay_jup = -GM_jup * dyJ / rJ**3

        # total acceleration
        ax = ax_sun + ax_jup
        ay = ay_sun + ay_jup

        # update (Euler-Cromer)
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

        t += dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

    xJ_list.append(aJ * np.cos(omega_J * t))
    yJ_list.append(aJ * np.sin(omega_J * t))

plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
```

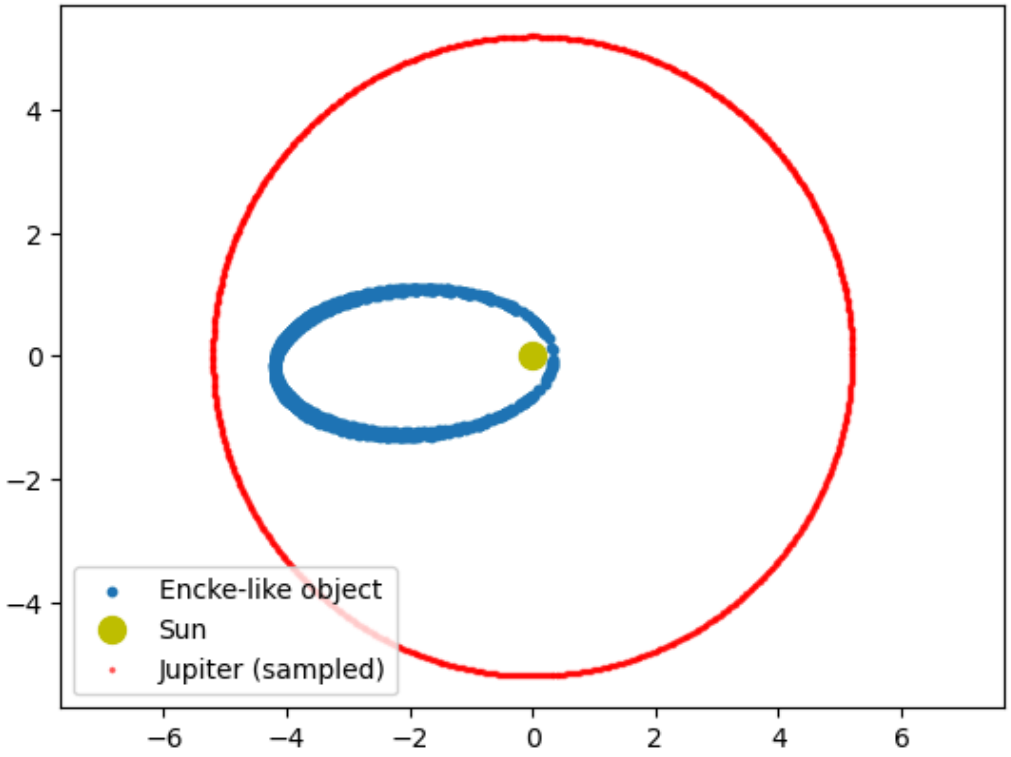
```
plt.plot(xJ_list, yJ_list, 'r.', markersize=3, alpha=0.5, label="Jupiter (sample
plt.axis("equal")
plt.legend()
plt.savefig('5b')
plt.show()

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```




```
Timestep = 0.00275
r min/max = 0.3310864441898541 4.207830857884208
r ratio = 12.709160799924872
Energy drift (max-min)/|E0| = 0.49104232222301925
Angular momentum drift (max-min)/|h0| = 0.022618885049297842
```

Extra testing for timescale

In [67]:
```
# reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
n_orbits = 8
n_snapshots_per_orbit = 30     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

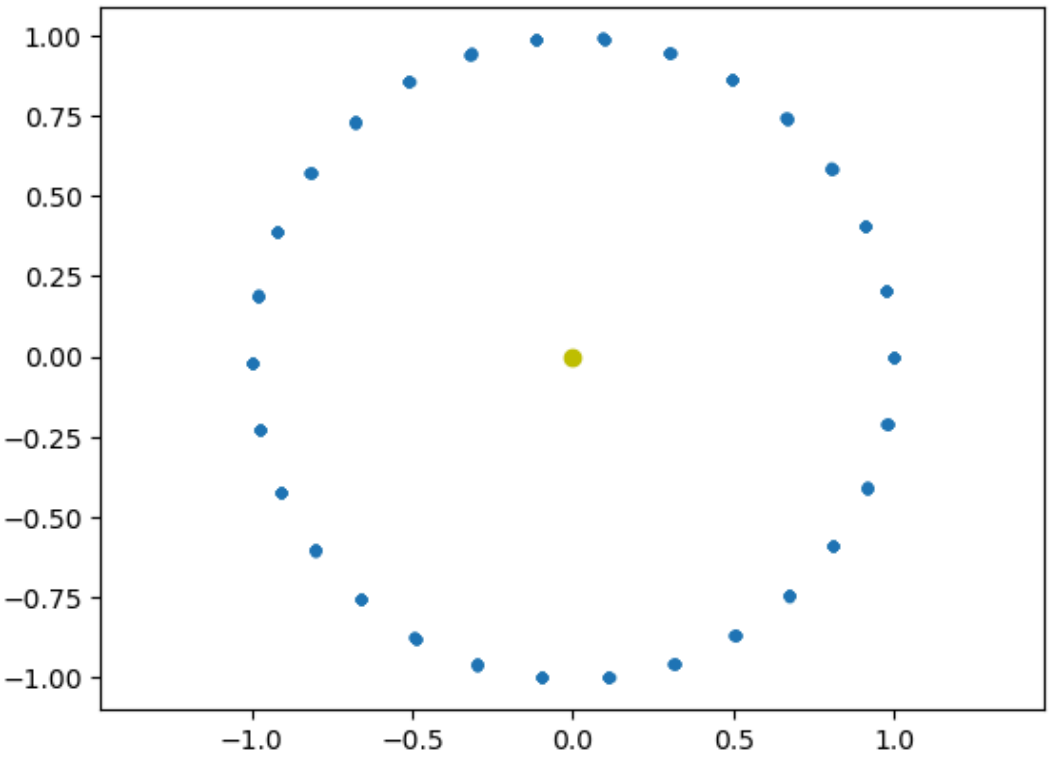
# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

plt.scatter(xs, ys, s=10)
plt.plot(0,0,'yo')
plt.axis("equal")
plt.show()
print('Timestep=',dt)
rs = np.hypot(xs, ys)
print("r min/max =", rs.min(), rs.max())
print('ratio=', rs.max() / rs.min())
rs = np.hypot(xs, ys)
vs2 = np.array(vxs)**2 + np.array(vys)**2
E = 0.5*vs2 - GM/rs

print("Energy drift (max-min)/|E0| =",
      (E.max() - E.min()) / abs(E[0]))
```




```
Timestep= 0.0016666666666666668
r min/max = 0.9948397537722348 1.0052702691633866
ratio= 1.0104846186047565
Energy drift (max-min)/|E0| = 0.00010962260461873541
```

In [68]:

```python
# reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = R**1.5

# You choose:
n_orbits = 8
n_snapshots_per_orbit = 60     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [],[]

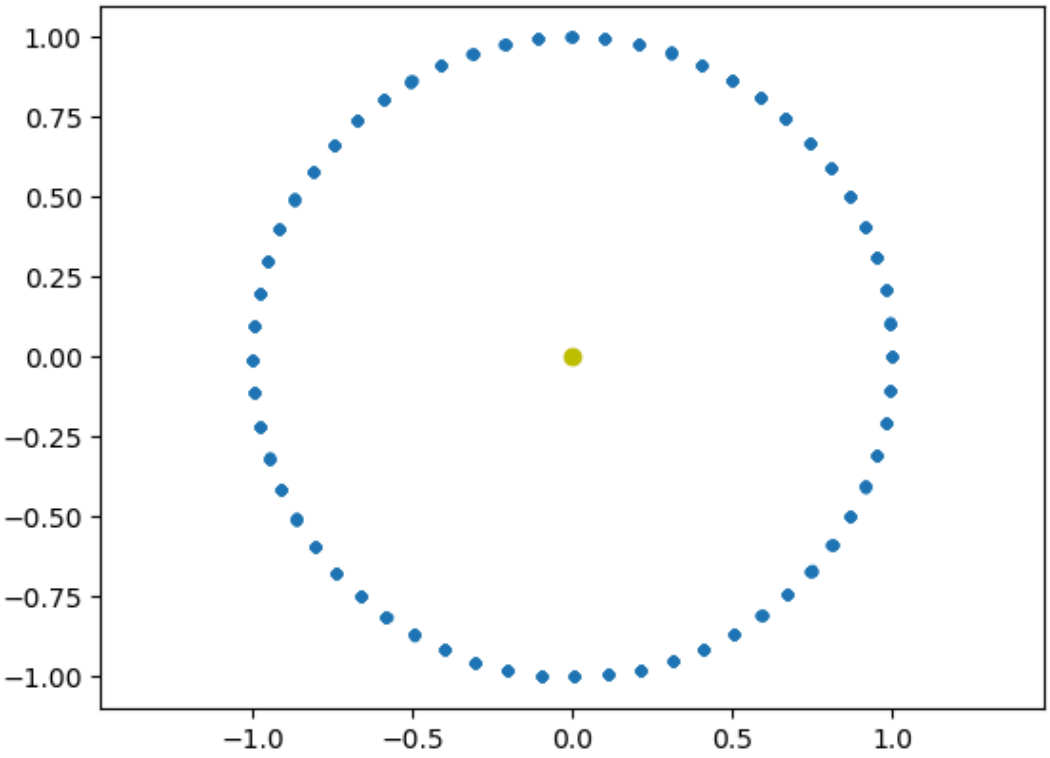
for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
```

```
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

plt.scatter(xs, ys, s=10)
plt.plot(0,0,'yo')
plt.axis("equal")
plt.show()
print('Timestep=',dt)
rs = np.hypot(xs, ys)
print("r min/max =", rs.min(), rs.max())
print('ratio=', rs.max() / rs.min())
rs = np.hypot(xs, ys)
vs2 = np.array(vxs)**2 + np.array(vys)**2
E = 0.5*vs2 - GM/rs

print("Energy drift (max-min)/|E0| =",
      (E.max() - E.min()) / abs(E[0]))
```




```
Timestep= 0.0008333333333333334
r min/max = 0.9973957746869052 1.0026316423229507
ratio= 1.005249538617395
Energy drift (max-min)/|E0| = 2.741291912140694e-05
```

In [69]:
```
# reset
x, y = 1.0, 0.0
vx, vy = 0.0, 2 * np.pi

# period from Kepler (need radius first)
R = np.hypot(x, y)
```

```
T = R**1.5

# You choose:
n_orbits = 8
n_snapshots_per_orbit = 120     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

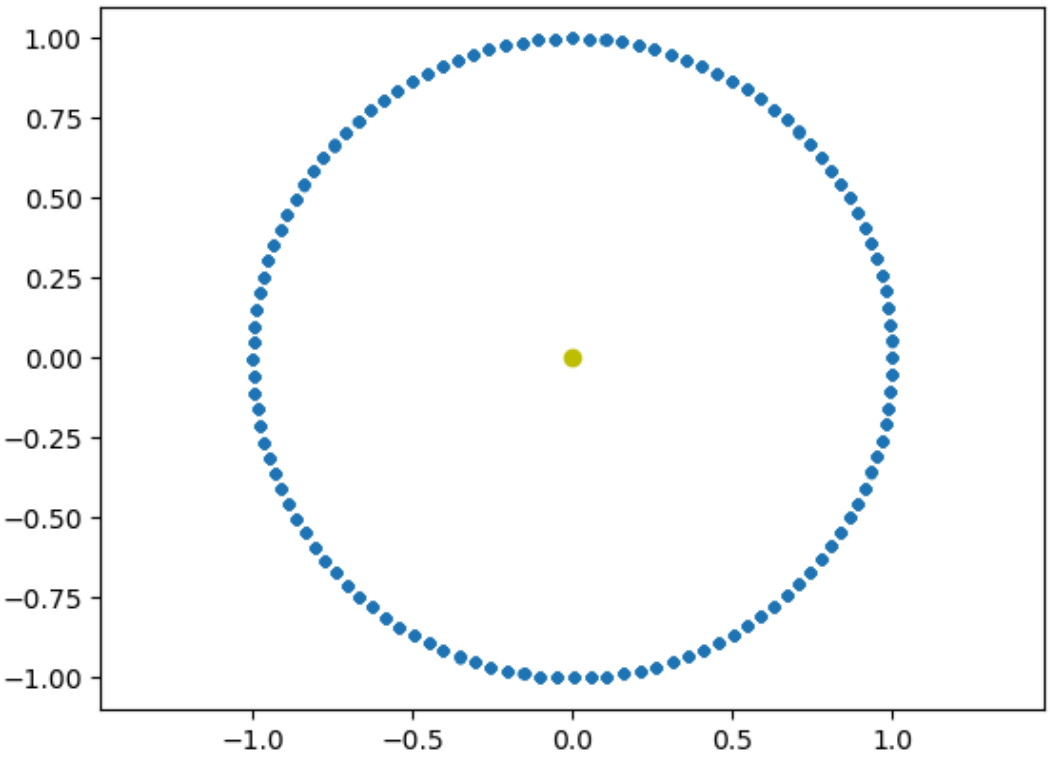
# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [],[]

for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

plt.scatter(xs, ys, s=10)
plt.plot(0,0,'yo')
plt.axis("equal")
plt.show()
print('Timestep=',dt)
rs = np.hypot(xs, ys)
print("r min/max =", rs.min(), rs.max())
print('ratio=', rs.max() / rs.min())
rs = np.hypot(xs, ys)
vs2 = np.array(vxs)**2 + np.array(vys)**2
E = 0.5*vs2 - GM/rs

print("Energy drift (max-min)/|E0| =",
      (E.max() - E.min()) / abs(E[0]))
```




```
Timestep= 0.0004166666666666667
r min/max = 0.9986944382800758 1.0013124157076791
ratio= 1.00262139982687
Energy drift (max-min)/|E0| = 6.853721105308742e-06
```

Testing for 5- different orbit values. 10, 30 B. Then 10, 30 A respectively

In [75]:

```python
import numpy as np
import matplotlib.pyplot as plt


# Encke-like initial conditions (planar approximation)
# Use orbital elements (approx): a = 2.218 AU, q = 0.339 AU (so e ~ 0.847)
# Start at perihelion on +x axis, velocity purely +y (tangential).

a_encke = 2.218   # AU
p_encke = 0.339   # AU

x, y = p_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/p_encke - 1.0/a_encke))  # perihelion speed (AU/

dt = 0.001
n_steps = 5000

# ----------------------------
# Jupiter parameters (2D circular orbit approximation)
# ----------------------------
aJ = 5.2
T_J = aJ**1.5
omega_J = 2*np.pi / T_J

GM_sun = GM
```

```
GM_jup = 0.0009543 * GM_sun

# period from Kepler
T = 3.3

# You choose:
n_orbits = 10
n_snapshots_per_orbit = 60
steps_between_snapshots = 20

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []
xJ_list, yJ_list = [], []

t = 0.0

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):

        # ---- Sun acceleration ----
        r = np.hypot(x, y)
        ax_sun = -GM_sun * x / r**3
        ay_sun = -GM_sun * y / r**3

        # ---- Jupiter position on circular orbit ----
        xJ = aJ * np.cos(omega_J * t)
        yJ = aJ * np.sin(omega_J * t)

        # ---- Jupiter acceleration ----
        dxJ, dyJ = x - xJ, y - yJ
        rJ = np.hypot(dxJ, dyJ)
        ax_jup = -GM_jup * dxJ / rJ**3
        ay_jup = -GM_jup * dyJ / rJ**3

        # total acceleration
        ax = ax_sun + ax_jup
        ay = ay_sun + ay_jup

        # update (Euler-Cromer)
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

        t += dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

    xJ_list.append(aJ * np.cos(omega_J * t))
    yJ_list.append(aJ * np.sin(omega_J * t))
```

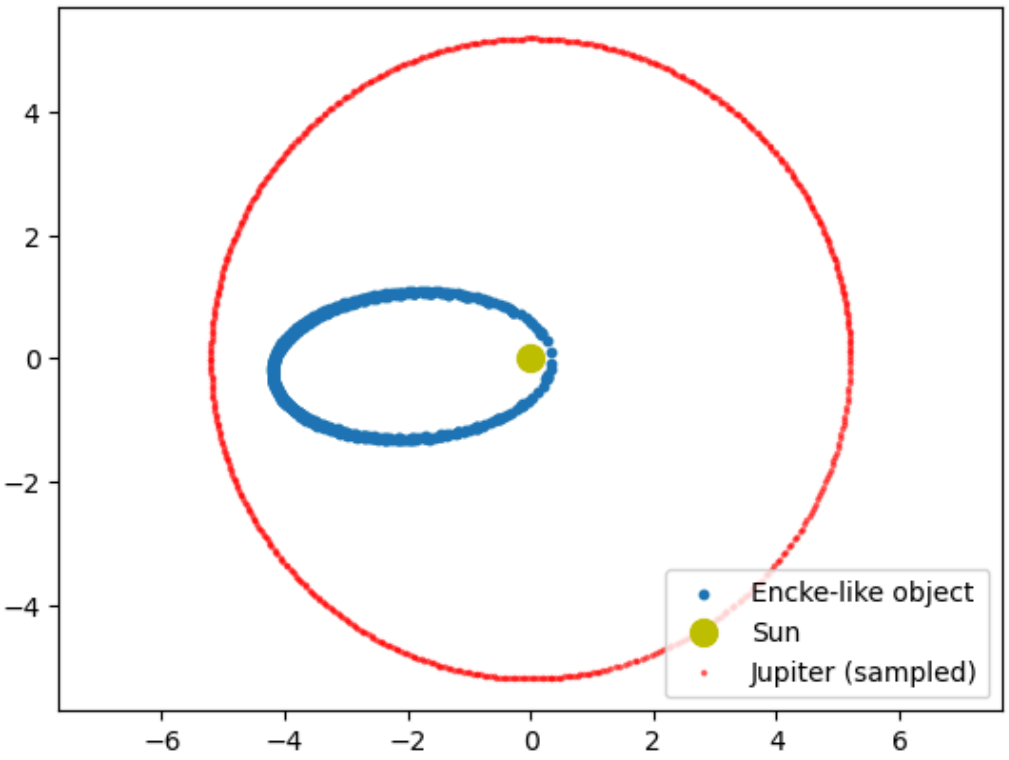
```
plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.plot(xJ_list, yJ_list, 'r.', markersize=3, alpha=0.5, label="Jupiter (sample
plt.axis("equal")
plt.legend()
plt.savefig('5b')
plt.show()

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```



```
Timestep = 0.00275
r min/max = 0.3434622809135162 4.207830857884208
r ratio = 12.251216776096992
Energy drift (max-min)/|E0| = 0.4673717377121843
Angular momentum drift (max-min)/|h0| = 0.013805276205746831
```

In [76]:

```python
import numpy as np
import matplotlib.pyplot as plt


# Encke-like initial conditions (planar approximation)
# Use orbital elements (approx): a = 2.218 AU, q = 0.339 AU (so e ~ 0.847)
# Start at perihelion on +x axis, velocity purely +y (tangential).

a_encke = 2.218   # AU
p_encke = 0.339   # AU

x, y = p_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/p_encke - 1.0/a_encke))  # perihelion speed (AU/

dt = 0.001
n_steps = 5000

# ----------------------------
# Jupiter parameters (2D circular orbit approximation)
# ----------------------------
aJ = 5.2
T_J = aJ**1.5
omega_J = 2*np.pi / T_J

GM_sun = GM
GM_jup = 0.0009543 * GM_sun

# period from Kepler
T = 3.3

# You choose:
n_orbits = 30
n_snapshots_per_orbit = 60
steps_between_snapshots = 20

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []
xJ_list, yJ_list = [], []

t = 0.0

for snap in range(n_snapshots):
    for _ in range(steps_between_snapshots):

        # ---- Sun acceleration ----
        r = np.hypot(x, y)
        ax_sun = -GM_sun * x / r**3
```

```
        ay_sun = -GM_sun * y / r**3

        # ---- Jupiter position on circular orbit ----
        xJ = aJ * np.cos(omega_J * t)
        yJ = aJ * np.sin(omega_J * t)

        # ---- Jupiter acceleration ----
        dxJ, dyJ = x - xJ, y - yJ
        rJ = np.hypot(dxJ, dyJ)
        ax_jup = -GM_jup * dxJ / rJ**3
        ay_jup = -GM_jup * dyJ / rJ**3

        # total acceleration
        ax = ax_sun + ax_jup
        ay = ay_sun + ay_jup

        # update (Euler-Cromer)
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

        t += dt

    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

    xJ_list.append(aJ * np.cos(omega_J * t))
    yJ_list.append(aJ * np.sin(omega_J * t))

plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.plot(xJ_list, yJ_list, 'r.', markersize=3, alpha=0.5, label="Jupiter (sample
plt.axis("equal")
plt.legend()
plt.savefig('5b')
plt.show()

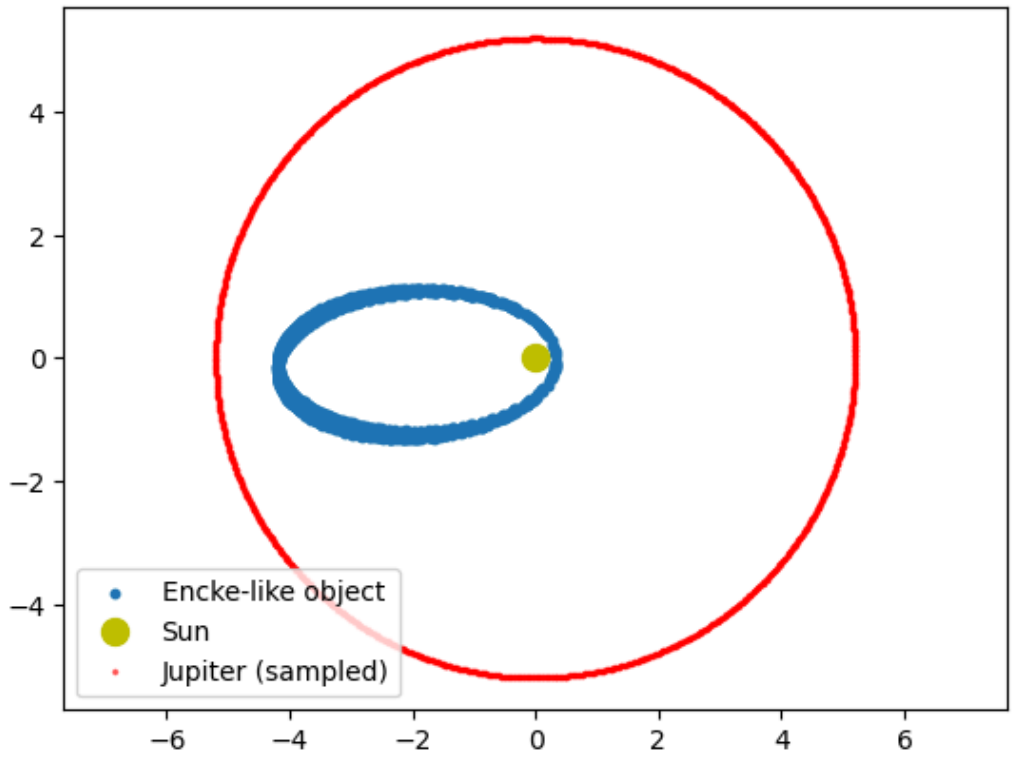
# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
```

```
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```




```
Timestep = 0.00275
r min/max = 0.3213559014295593 4.207830857884208
r ratio = 13.093989682982553
Energy drift (max-min)/|E0| = 0.5078954405407342
Angular momentum drift (max-min)/|h0| = 0.03215745410316601
```

In [77]:
```
a_encke = 2.218   # AU
q_encke = 0.339   # AU

x, y = q_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/q_encke - 1.0/a_encke))  # perihelion speed (AU/
dt = 0.001
n_steps = 5000

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = 3.3

# You choose:
n_orbits = 10
n_snapshots_per_orbit = 60     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []
```

```
for snap in range(n_snapshots):
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

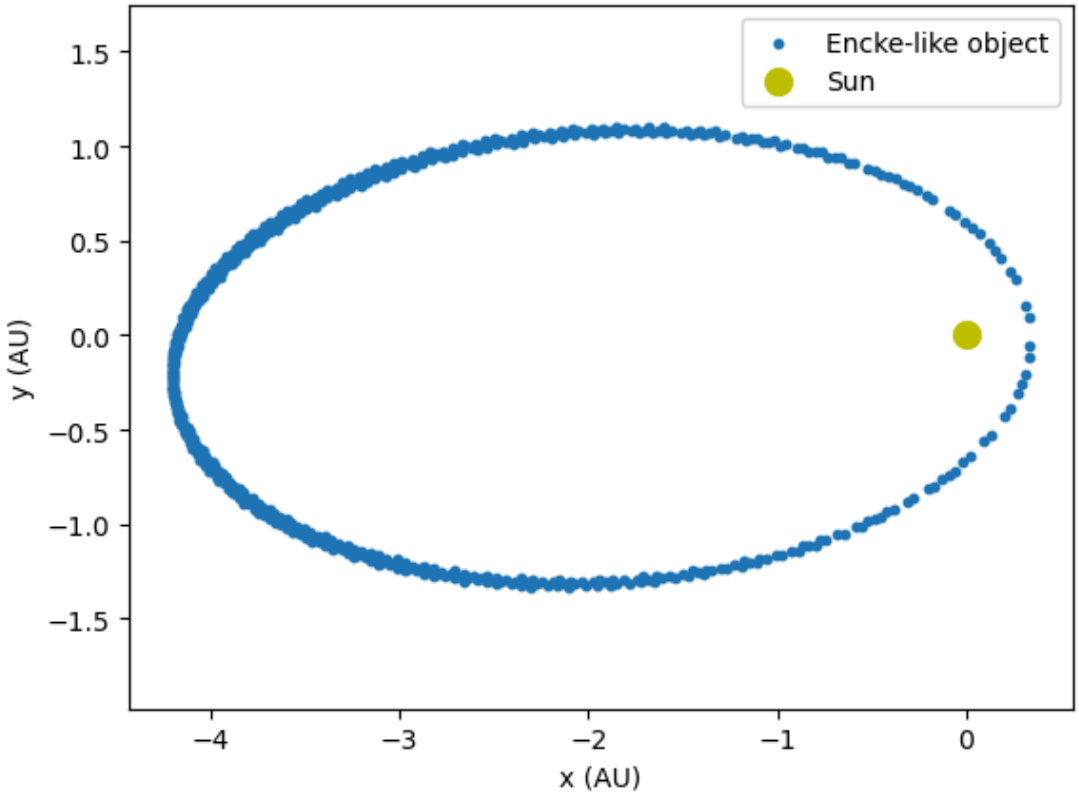
plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.axis("equal")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('5a')
plt.show()

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```




```
Timestep = 0.00275
r min/max = 0.34290289064756746 4.20581779378426
r ratio = 12.265331989011905
Energy drift (max-min)/|E0| = 0.4476921269330607
Angular momentum drift (max-min)/|h0| = 8.574514234371761e-15
```

In [78]:

```
a_encke = 2.218   # AU
q_encke = 0.339   # AU

x, y = q_encke, 0.0
vx, vy = 0.0, np.sqrt(GM * (2.0/q_encke - 1.0/a_encke))  # perihelion speed (AU/
dt = 0.001
n_steps = 5000

# period from Kepler (need radius first)
R = np.hypot(x, y)
T = 3.3

# You choose:
n_orbits = 30
n_snapshots_per_orbit = 60     # how many plotted points per orbit
steps_between_snapshots = 20      # inner integration steps per plotted point

# Total snapshots you will store/plot
n_snapshots = n_orbits * n_snapshots_per_orbit

# dt chosen so that one orbit lasts T and contains (n_snapshots_per_orbit * step
dt = T / (n_snapshots_per_orbit * steps_between_snapshots)

# storage for snapshots (must be outside the loop)
xs, ys = [], []
vxs, vys = [], []

for snap in range(n_snapshots):
```

```
    # only do the desired number of steps between snapshots
    for _ in range(steps_between_snapshots):
        r = np.hypot(x, y)
        ax = -GM * x / r**3
        ay = -GM * y / r**3
        vx += ax * dt
        vy += ay * dt
        x += vx * dt
        y += vy * dt

    # record one snapshot point
    xs.append(x)
    ys.append(y)
    vxs.append(vx)
    vys.append(vy)

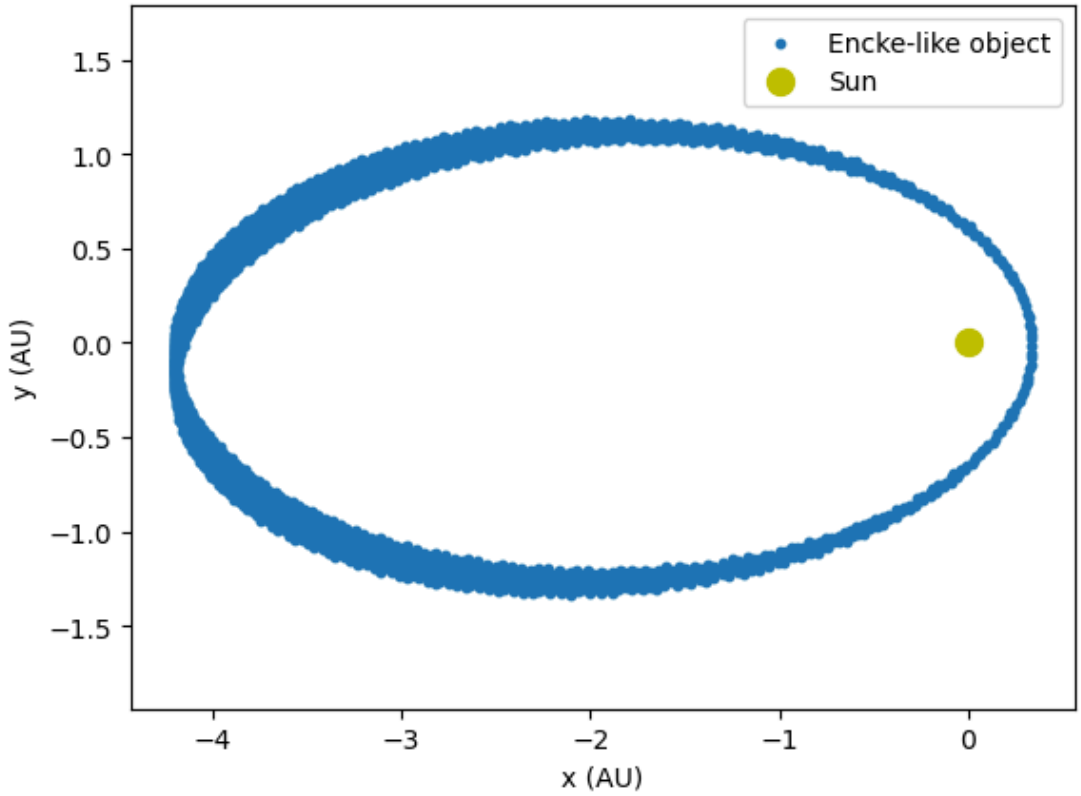
plt.scatter(xs, ys, s=10, label="Encke-like object")
plt.plot(0, 0, 'yo', markersize=10, label="Sun")
plt.axis("equal")
plt.xlabel("x (AU)")
plt.ylabel("y (AU)")
plt.legend(loc="upper right")
plt.savefig('5a')
plt.show()

# convert to arrays once
xs = np.array(xs)
ys = np.array(ys)
vxs = np.array(vxs)
vys = np.array(vys)

# radial diagnostics
rs = np.hypot(xs, ys)
print("Timestep =", dt)
print("r min/max =", rs.min(), rs.max())
print("r ratio =", rs.max() / rs.min())

# energy
vs2 = vxs**2 + vys**2
E = 0.5 * vs2 - GM / rs
rel_E_drift = (E.max() - E.min()) / abs(E[0])
print("Energy drift (max-min)/|E0| =", rel_E_drift)

# angular momentum
h = xs * vys - ys * vxs
rel_h_drift = (h.max() - h.min()) / abs(h[0])
print("Angular momentum drift (max-min)/|h0| =", rel_h_drift)
```




```
Timestep = 0.00275
r min/max = 0.338787734203839 4.205819463961515
r ratio = 12.414320352669534
Energy drift (max-min)/|E0| = 0.4480005599793379
Angular momentum drift (max-min)/|h0| = 1.9292657027336464e-14
```

In [ ]: